

Autosoaring PX4 Firmware — Algorithm Specifications

Additions to Existing Modules (red = new code)

Radhouene

May 5, 2026

Contents

1	Overview	2
2	Module 1: TECS (Total Energy Control System)	2
2.1	M1 — Altitude Reference Freeze (<code>TECS::update</code>)	2
2.2	M2 — Gravity-Limited Acceleration Cap (<code>_calcAirspeedControlOutput</code>)	3
2.3	M3 — Underspeed Detection Bypass (<code>_detectUnderspeed</code>)	3
2.4	M4 — Energy Weighting Override (<code>_updateSpeedAltitudeWeights</code>)	3
2.5	M5 — Throttle Hard-Zero (<code>_calcThrottleControlOutput</code>)	4
2.6	M6 — Throttle Integrator Exponential Decay (<code>_calcThrottleControlUpdate</code>)	4
2.7	M7 — Load-Factor STE Correction Disabled (<code>_calcThrottleControlSteRate</code>)	4
2.8	M8 — Pitch Integrator Partial Reset (<code>_calcPitchControl</code>)	5
2.9	M9 — Glide Airspeed Setpoint (<code>calcTrueAirspeedSetpoint</code>)	5
2.10	M10 — Fast-Descend Suppression (<code>_setFastDescend</code>)	5
3	Module 2: FixedwingPositionControl	6
3.1	C1 — DDS Poll and Authority Arbitration (<code>Run()</code>)	6
3.2	C2 — CLI Soaring Commands (<code>vehicle_command_poll()</code>)	7
3.3	C3 — Eager Glide Airspeed Push (<code>parameters_update()</code>)	8
3.4	C4 — Soaring Dispatch Block (<code>control_auto()</code>)	9
3.5	C6 — Symmetric Throttle Ramps	10
4	Module 3: Commander	11
4.1	Commander — CLI Soaring Sub-commands	11
4.2	Commander — DO_REPOSITION Fix	11
5	Module 4: Navigator	12
5.1	MissionBase — Mission Rewind Prevention	12
5.2	MissionBase — Pause / Resume Sequence Tracking	12
6	New uORB Messages	13
7	New Parameters	13

1 Overview

This document presents pseudocode algorithms for all PX4 modules modified as part of the autonomous soaring extension. **Black text** describes stock PX4 behaviour; **red text** marks every function, block, or variable that was added or changed.

Four modules are covered:

1. **TECS** (`src/lib/tecs/TECS.cpp/.hpp`) — 10 modifications enabling engine-off glide control.
2. **FixedwingPositionControl** (`src/modules/fw_pos_control/`) — 6 changes implementing the soaring dispatch loop, authority arbitration, altitude safety, and NPFG bypass.
3. **Commander** (`src/modules/commander/Commander.cpp`) — CLI soaring sub-commands and DO_REPOSITION fix.
4. **Navigator** (`src/modules/navigator/`) — mission-rewind prevention and clean activation.

2 Module 1: TECS (Total Energy Control System)

TECS controls throttle and pitch to track energy setpoints. All **red** modifications add a *gliding mode* where the engine is off and pitch controls airspeed instead of altitude.

2.1 M1 — Altitude Reference Freeze (TECS::update)

Algorithm 1: TECS altitude reference model selection

```
1 Function TECS::update(dt, altitude, EAS_setpoint, ...):  
    // ... filter airspeed, compute eas_to_tas ...  
2    if gliding_mode_enabled then ▷ new: glide mode freezes altitude ref each cycle  
3        frozen_state ← {alt = altitude, alt_rate = hgt_rate}  
4        altitude_reference_model.initialize(frozen_state)  
5    end  
6    else if fast_descend ≈ 1 then  
7        // stock: reset during fast descent  
        altitude_reference_model.initialize(init_state)  
8    end  
9    else  
10       altitude_reference_model.update(dt, ...)  
11    end  
12    tas_sp ← calcTrueAirspeedSetpoint(eas_to_tas, EAS_setpoint)  
    // ... run TECSControl ...  
13 end
```

2.2 M2 — Gravity-Limited Acceleration Cap (`_calcAirspeedControlOutput`)

Algorithm 2: Airspeed rate setpoint cap — glide uses gravity bound

```
1 Function _calcAirspeedControlOutput(input, limit, param, flag):  
2   if flag.airspeed_enabled then  
3     available_accel  $\leftarrow$  if gliding_mode_enabled then |STE_rate_min| else  
       STE_rate_max  
4      $\triangleright$  gravity is the only source of kinetic energy gain in glide  
5     max_tas_rate_sp  $\leftarrow$   $(\text{fast\_descend} \cdot 0.5 + 0.5) \times \text{available\_accel} / \text{tas}$   
6     airspeed_rate_output  $\leftarrow$  clamp((tas_sp - tas)  $\times$  gain, min_tas_rate_sp,  
       max_tas_rate_sp)  
7   end  
8 end
```

2.3 M3 — Underspeed Detection Bypass (`_detectUnderspeed`)

Algorithm 3: Underspeed detection — bypassed in glide

```
1 Function _detectUnderspeed(input, limit, param, flag):  
2   if gliding_mode_enabled then  $\triangleright$  glide speed is below cruise trim; bypass falsely  
     triggers  
3     _ratio_undersped  $\leftarrow$  0  
4     return  
5   end  
   // stock underspeed detection using cruise trim boundary ...  
6   tas_error_bound  $\leftarrow$  tas_error_percentage  $\times$  equiv_airspeed_trim  
7   _ratio_undersped  $\leftarrow$  clamp(...)  
8 end
```

2.4 M4 — Energy Weighting Override (`_updateSpeedAltitudeWeights`)

Algorithm 4: Speed/altitude weighting — full speed-on-elevator in glide

```
1 Function _updateSpeedAltitudeWeights(param, flag):  
2   if gliding_mode_enabled then  $\triangleright$  pitch controls airspeed; altitude follows  
     naturally  
3     weight.spe_weighting  $\leftarrow$  0  
4     weight.ske_weighting  $\leftarrow$  2  
5     return weight  
6   end  
   // stock weighting from pitch_speed_weight parameter ...  
7   pitch_speed_weight  $\leftarrow$  clamp(param.pitch_speed_weight, 0, 2)  
8   weight.ske_weighting  $\leftarrow$  ...  
9   return weight  
10 end
```

2.5 M5 — Throttle Hard-Zero (`_calcThrottleControlOutput`)

Algorithm 5: Throttle output — hard zero in glide

```
1 Function _calcThrottleControlOutput(limit, ste_rate, param, flag):  
2   if gliding_mode_enabled then ▷ engine must be off; return throttle_min directly  
3     | return param.throttle_min  
4   end  
   // stock PI throttle output ...  
5   STE_rate_to_thr  $\leftarrow 1 / (\text{STE\_rate\_max} - \text{STE\_rate\_min})$   
6   throttle_sp  $\leftarrow \dots$   
7   return clamp(throttle_sp, throttle_min, throttle_max)  
8 end
```

2.6 M6 — Throttle Integrator Exponential Decay (`_calcThrottleControlUpdate`)

Algorithm 6: Throttle integrator — exponential decay in glide

```
1 Function _calcThrottleControlUpdate(dt, limit, ste_rate, param, flag):  
2   if gliding_mode_enabled then ▷ decay instead of wind-up; avoids spike on power  
   restore  
3     | decay  $\leftarrow \max(\text{param.glide\_i\_decay}, 0.1)$   
4     | _throttle_integ_state  $\leftarrow \text{\_throttle\_integ\_state} - dt \times \text{\_throttle\_integ\_state} /$   
   decay  
5     | return  
6   end  
   // stock PI integrator accumulation ...  
7   STE_rate_error  $\leftarrow \text{ste\_rate.setpoint} - \text{ste\_rate.estimate}$   
8   _throttle_integ_state  $+= \dots$   
9 end
```

2.7 M7 — Load-Factor STE Correction Disabled (`_calcThrottleControlSteRate`)

Algorithm 7: STE rate setpoint — load factor skipped in glide

```
1 Function _calcThrottleControlSteRate(limit, sp_rates, param, flag):  
2   if NOT gliding_mode_enabled then ▷ throttle is zero anyway; correction only  
   pollutes debug signal  
3     | ste_rate.setpoint  $+= \text{load\_factor\_correction} \times (\text{load\_factor} - 1)$   
4   end  
5   ste_rate.setpoint  $\leftarrow \text{clamp}(\text{ste\_rate.setpoint}, \text{STE\_rate\_min}, \text{STE\_rate\_max})$   
6   return ste_rate  
7 end
```

2.8 M8 — Pitch Integrator Partial Reset (`_calcPitchControl`)

Algorithm 8: Pitch control — symmetric integrator reset at glide transitions

```
1 Function _calcPitchControl(dt, input, sp_rates, param, flag):
2   if gliding_mode_enabled AND NOT _prev_gliding_mode then ▷ entry: cruise bias
      fights speed SP; halve to reduce overshoot
3     | _pitch_integ_state ← _pitch_integ_state × 0.5
4   end
5   else if NOT gliding_mode_enabled AND _prev_gliding_mode then ▷ exit:
      speed-control bias causes pitch spike; damp symmetrically
6     | _pitch_integ_state ← _pitch_integ_state × 0.5
7   end
8   _prev_gliding_mode ← gliding_mode_enabled
9   weight ← _updateSpeedAltitudeWeights(param, flag)
      // ... SEB rate, integrator update, pitch output, rate limit ...
10 end
```

2.9 M9 — Glide Airspeed Setpoint (`calcTrueAirspeedSetpoint`)

Algorithm 9: Airspeed setpoint — glide path bypasses powered-flight minimum

```
1 Function TECS::calcTrueAirspeedSetpoint(eas_to_tas, eas_setpoint):
2   if gliding_mode_enabled then ▷ use pre-computed glide EAS; do not clamp to
      FW_AIRSPD_MIN
3     | if gliding_airspeed_setpoint > 0 AND isfinite then
4       | tas_sp ← eas_to_tas × gliding_airspeed_setpoint
5       | tas_stall ← eas_to_tas × glide_stall_eas
          ▷ floor = FW_AIRSPD_STALL converted to TAS, not FW_AIRSPD_MIN
6       | glide_floor ←  $\max(\text{tas\_stall}, \text{tas\_min} \times \sqrt{n_z} \times 0.5)$ 
7       | return clamp(tas_sp, glide_floor, tas_max)
8     | end
9     | return clamp(eas_to_tas × equiv_airspeed_trim, tas_min, tas_max)
10  end
11  return lerp(eas_to_tas × eas_setpoint, tas_max, fast_descend)
12 end
```

2.10 M10 — Fast-Descend Suppression (`_setFastDescend`)

Algorithm 10: Fast descend — suppressed entirely in glide

```
1 Function TECS::_setFastDescend(alt_setpoint, alt):
2   if gliding_mode_enabled then ▷ glide is intentionally below alt SP; suppress
      fast-descend
3     | _fast_descend ← 0
4     | _enabled_fast_descend_timestamp ← 0
5     | return
6   end
      // stock: activate when alt > alt_setpoint + alt_err_fd ...
7 end
```

3 Module 2: FixedwingPositionControl

3.1 C1 — DDS Poll and Authority Arbitration (Run())

Algorithm 11: DDS poll: inhibit, CLI override, staleness watchdog

```
1 Function FixedwingPositionControl::Run():
  // ... update local position, parameters ...
2  // -- ADDED: DDS soaring poll --
3  if autosoaring_control_sub.update(soaring_msg) then
4    now ← hrt_absolute_time()
5    dds_inhibited ← (dds_inhibit_until_us > 0 AND now < dds_inhibit_until_us)
6    dds_wants_soaring ← (soaring_msg.soaring_mode ≠ OFF)
7    if dds_inhibited AND dds_wants_soaring then           ▷ soar:off cooldown active;
      ignore re-enable
      | // drop message
8    end
9    else
10     if NOT dds_wants_soaring then
11       | dds_inhibit_until_us ← 0
12     end
13     if NOT _soaring_local_override then           ▷ CLI lock? then DDS must not
       overwrite
14       | prev_thermal ← (autosoaring_control.mode ∈ {3,4})
15       | new_thermal ← (soaring_msg.mode ∈ {3,4})
16       | _autosoaring_control ← soaring_msg
17       | autosoaring_last_recv_us ← now
18       if NOT prev_thermal AND new_thermal then   ▷ thermal just activated:
         reset loiter anchor + debounce
19       | soaring_mode_cmd_last_us ← 0
20       | soaring_last_lat/lon ← NAN
21     end
22   end
23 end
24 end
  // -- Staleness watchdog (DDS path only) --
25 if NOT _soaring_local_override AND autosoaring_last_recv_us > 0 AND elapsed > 2 s
    then
26   | _autosoaring_control.soaring_mode ← OFF
27   | PX4_WARN("companion silent >2 s")
28 end
29 vehicle_command_poll()
  // ... set_control_mode_current, dispatch switch ...
30 end
```

3.2 C2 — CLI Soaring Commands (vehicle_command_poll())

Algorithm 12: vehicle_command_poll — CUSTOM_0 and DO_SET_MODE

```
1 Function vehicle_command_poll():
2   while vehicle_command_sub.update(cmd) do
3     if cmd.command = DO_GO_AROUND then
4       | ...
5     end
6     // stock
7     else if cmd.command = DO_CHANGE_SPEED then
8       | ...
9     end
10    // stock
11    else if cmd.command = CUSTOM_0 then                                ▷ ADDED: soaring CLI path
12      new_mode ← clamp((int)cmd.param1, 0, 4)
13      _autosoaring_control.soaring_mode ← new_mode
14      if cmd.param3 > 0 then
15        | _autosoaring_control.loiter_radius_m ← cmd.param3
16      end
17      if isfinite(cmd.param2) AND cmd.param2 > 0 then
18        | _autosoaring_control.bank_angle_cmd ← cmd.param2
19      end
20      enabling ← (new_mode ≠ 0)
21      if enabling then
22        | _soaring_local_override ← true
23        | dds_inhibit_until_us ← 0
24        | soaring_forbidden_latched ← false
25        | autosoaring_last_rcv_us ← 0
26        | _soaring_expect_set_mode ← true
27        | soaring_mode_cmd_last_us ← 0
28        | soaring_last_lat/lon ← NAN
29        | publish_status(SOURCE_CLI_ON, new_mode)
30      end
31      else
32        | _soaring_local_override ← false
33        | dds_inhibit_until_us ← now + 5 s
34        | autosoaring_last_rcv_us ← 0
35        | tecs.set_gliding_mode_enabled(false)
36        | publish_status(SOURCE_CLI_OFF, dds_inhibit_until_us)
37      end
38      end
39      else if cmd.command = DO_SET_MODE AND _soaring_local_override then
40        ▷ ADDED: filter QGC heartbeats / detect soaring exit
41        if _soaring_expect_set_mode then
42          | _soaring_expect_set_mode ← false
43          | // consume pairing
44        end
45        else
46          target_ok ← (target ∈ {AUTO_MISSION, AUTO_LOITER})
47          if NOT target_ok then
48            | soaring_mode ← OFF
49            | _soaring_local_override ← false
50            | tecs.set_gliding_mode_enabled(false)
51          end
52          // else: QGC heartbeat, keep soaring active
53        end
54      end
55    end
56  end
```

3.3 C3 — Eager Glide Airspeed Push (parameters_update())

Algorithm 13: parameters_update — push glide airspeed to TECS eagerly

```
1 Function parameters_update():  
    // ... stock TECS and NPFG parameter setters ...  
    // -- ADDED: soaring parameter push --  
2    polar_a ← FW_POLAR_A.get()  
3    polar_b ← FW_POLAR_B.get()  
4    v_polar ← (polar_a > 0 AND polar_b > 0) ?  $\sqrt{b/a}$  : 0  
5    if mode = GLIDE_POLAR AND v_polar > 0 then  
6        | tecs.set_gliding_airspeed_setpoint(v_polar)  
7    else  
8        | tecs.set_gliding_airspeed_setpoint(FW_GLIDE_AIRSPD)  
9    end  
10   tecs.set_glide_i_decay(FW_GLIDE_I_DECAY)  
11   tecs.set_glide_stall_eas(FW_AIRSPD_STALL)  
12 end
```

3.4 C4 — Soaring Dispatch Block (control_auto())

Algorithm 14: control_auto — full soaring dispatch (9 steps)

```
1 Function control_auto(...):
    // -- ADDED: soaring pre-dispatch block --
    // Step 1: extract mode variables
2    soaring_cmd ← _autosoaring_control.soaring_mode
3    glide_cmd ← (mode ∈ {1,2})
4    thermal_cmd ← (mode ∈ {3,4})
5    soaring_allowed ← (soaring_cmd ≠ OFF) AND (override OR NOT latched)
    // Step 2: altitude floor -- FW_ALT_MIN (enforced on ALL paths)
6    if soaring_requested AND altitude < FW_ALT_MIN then
7        if NOT soaring_forbidden_latched then
8            soaring_forbidden_latched ← true
9            if _soaring_local_override then
10                _soaring_local_override ← false
11                PX4_WARN("CLI override released: below FW_ALT_MIN")
12            end
13        end
14    end
    // Step 3: altitude ceiling FW_ALT_MAX with hysteresis
15    alt_max_reached ← update_hysteresis(altitude, FW_ALT_MAX, FW_ALT_HYST)
    // Step 4: effective mode flags
16    effective_glide ← soaring_allowed AND (glide_cmd OR alt_max_reached)
17    effective_thermal ← soaring_allowed AND thermal_cmd AND NOT alt_max_reached
18    effective_thermal_bank ← effective_thermal AND (mode = 4)
    // Step 5: TECS flag and airspeed injection
19    tecs.set_gliding_mode_enabled(effective_glide OR effective_thermal)
20    if mode = 1 AND airspeed_cmd > 0 then
21        | tecs.set_gliding_airspeed_setpoint(airspeed_cmd)
22    else
23        if mode = 2 then
24            | tecs.set_gliding_airspeed_setpoint( $\sqrt{b/a}$ )
25        else
26            | tecs.set_gliding_airspeed_setpoint(FW_GLIDE_AIRSPD)
27        end
28    end
    // Step 6: thermal DO_REPOSITION (first activation or position change)
29    if effective_thermal AND pos_changed then
30        | publish VEHICLE_CMD_DO_REPOSITION(lat, lon, altitude, signed_radius)
31    end
    // Step 7: glide DO_SET_MODE heartbeat (1 s period)
32    if effective_glide AND elapsed > 1 s then
33        | publish VEHICLE_CMD_DO_SET_MODE(AUTO_MISSION)
34    end
    // Step 8: thermal exit
35    if soaring_was_thermal AND NOT effective_thermal AND NOT effective_glide then
36        | publish VEHICLE_CMD_DO_SET_MODE(AUTO_MISSION)
37    end
    // Step 9: glide exit
38    if soaring_was_glide AND NOT effective_glide AND NOT effective_thermal then
39        | publish VEHICLE_CMD_DO_SET_MODE(AUTO_MISSION)
40        | _soaring_local_override ← false
41    end
42    soaring_was_glide ← effective_glide
43    soaring_was_thermal ← effective_thermal9
    // -- ADDED: Mode 4 NPFG bypass (C5) --
```

3.5 C6 — Symmetric Throttle Ramps

Algorithm 15: Throttle ramp on glide entry and exit

```
1 Function throttle_ramp_logic():
2   // -- ADDED: symmetric entry/exit ramp --
3   if effective_soaring AND NOT soaring_was_active then
4     // Glide entry: cache thrust and start ramp-down
5     soaring_pre_glide_thrust  $\leftarrow$  current_thrust
6     soaring_entry_ramp_start_us  $\leftarrow$  now
7   end
8   if NOT effective_soaring AND soaring_was_active then
9     // Glide exit: start ramp-up
10    soaring_ramp_start_us  $\leftarrow$  now
11  end
12  // Entry ramp: linear interpolation pre_glide_thrust  $\rightarrow$  0
13  if soaring_entry_ramp_start_us > 0 then
14    alpha  $\leftarrow$  elapsed / FW_GLIDE_RAMP_T
15    att_sp.thrust  $\leftarrow$  lerp(soaring_pre_glide_thrust, 0, alpha)
16  end
17  // Exit ramp: 0  $\rightarrow$  normal TECS throttle
18  if soaring_ramp_start_us > 0 then
19    alpha  $\leftarrow$  elapsed / FW_GLIDE_RAMP_T
20    att_sp.thrust  $\leftarrow$  lerp(0, tecs_thrust, alpha)
21  end
22 end
```

4 Module 3: Commander

4.1 Commander — CLI Soaring Sub-commands

Algorithm 16: Commander main() — soaring CLI sub-commands

```
1 Function Commander::main(argc, argv):
  // ... existing mode / calibration commands ...
2  // -- ADDED: soaring sub-commands --
3  if argv[1] = "soar:glide" then
4    send_vehicle_command(CUSTOM_0, param1=1.0)
5    send_vehicle_command(DO_SET_MODE, AUTO_MISSION)
6  end
7  else if argv[1] = "soar:polar" then
8    send_vehicle_command(CUSTOM_0, param1=2.0)
9    send_vehicle_command(DO_SET_MODE, AUTO_MISSION)
10 end
11 else if argv[1] = "soar:thermal" then
12   send_vehicle_command(CUSTOM_0, param1=3.0)
13   send_vehicle_command(DO_SET_MODE, AUTO_LOITER)
14 end
15 else if argv[1] = "soar:bank" then
16   send_vehicle_command(CUSTOM_0, param1=4.0)
17   send_vehicle_command(DO_SET_MODE, AUTO_LOITER)
18 end
19 else if argv[1] = "soar:off" then
20   send_vehicle_command(CUSTOM_0, param1=0.0)
21   send_vehicle_command(DO_SET_MODE, AUTO_MISSION)
22 end
23 end
```

4.2 Commander — DO_REPOSITION Fix

Algorithm 17: Commander — DO_REPOSITION param2=0 now accepted

```
1 Function handle_command(cmd):
2  if cmd = DO_REPOSITION then
3    mode_switch_not_requested ← (param2 AND 0x1 = 0)
4    unsupported_bits_set ← (param2 AND 0xFFFE ≠ 0)
5    if mode_switch_not_requested then ▷ ADDED: accept position-only reposition
6      (param2=0)
7      | cmd_result ← ACCEPTED
8    end
9    else if unsupported_bits_set then
10     | answer_command(UNSUPPORTED)
11   end
12   else
13     | // stock: normal mode-switch + reposition
14   end
15 end
```

5 Module 4: Navigator

5.1 MissionBase — Mission Rewind Prevention

Algorithm 18: MissionBase::updateMavlinkMission — stale uORB guard

```
1 Function MissionBase::updateMavlinkMission(new_mission):
2   mission_items_changed  $\leftarrow$  (new_mission.mission_id  $\neq$  _mission.mission_id)
3   // -- ADDED: prevent stale uORB from rewinding mission progress --
4   if NOT isActive() AND _mission_paused_seq  $\geq$  0 AND NOT mission_items_changed
      AND new_mission.dataman_id = _mission.dataman_id AND new_mission.current_seq
      < _mission_paused_seq then
5     new_mission.current_seq  $\leftarrow$  _mission_paused_seq
      ▷ Clamp stale QGC seq to the value at pause
6   end
7   _mission  $\leftarrow$  new_mission
8   if data changed then
9     onMissionUpdate(...)
10  end
11 end
```

5.2 MissionBase — Pause / Resume Sequence Tracking

Algorithm 19: MissionBase on_inactivation and on_activation

```
1 Function MissionBase::on_inactivation():
2   // ... stock: disable camera trigger, gimbal, etc. ...
3   _mission_paused_seq  $\leftarrow$  _mission.current_seq
4   ▷ Snapshot sequence on exit (e.g. entering AUTO_LOITER for thermal)
5 end
6 Function MissionBase::on_activation():
7   // ... stock: check mission restart, update mission, fill cache ...
8   _mission_paused_seq  $\leftarrow$  -1
9   ▷ Guard no longer needed once active
10  _navigator  $\rightarrow$  reset_triplets()
11  ▷ Clear loiter setpoint so mission starts with a fresh triplet
12 end
```

6 New uORB Messages

Algorithm 20: AutosoaringControl.msg — ROS2 → FMU command

```

1 uint8 soaring_mode    ▷ 0=OFF 1=GLIDE_FIXED 2=GLIDE_POLAR 3=THERMAL_LOITER
   4=THERMAL_BANK
2
3 float32 airspeed_cmd  ▷ Mode 1 EAS override [m/s]; NaN = use FW_GLIDE_AIRSPD
4
5 float32 bank_angle_cmd ▷ Thermal bank angle [deg]
6
7 float32 loiter_radius_m ▷ Thermal loiter radius [m]; NaN = auto
8
9 float32 loiter_lat, loiter_lon ▷ Thermal centre; 0/0 = hover here
10
11 bool loiter_clockwise ▷ Turn direction
12

```

Algorithm 21: AutosoaringStatus.msg — FMU → ROS2 feedback

```

1 uint8 source    ▷ 0=DDS 1=CLI_OFF 2=watchdog 3=CLI_ON
2
3 uint8 soaring_mode_effective ▷ Mode actually applied
4
5 uint64 dds_inhibit_until_us ▷ DDS inhibit expiry; 0 = no inhibit
6
7 uint8 fmu_soaring_phase ▷ 0=CRUISE 1=GLIDING 2=SEARCHING 3=CENTERING 4=THERMAL
8

```

7 New Parameters

Parameter	Default	Unit	Purpose
FW_GLIDE_AIRSPD	10.0	m/s	Fixed EAS for SOARING_GLIDE_FIXED and thermal hold
FW_ALT_MIN	100.0	m	Safety floor — soaring disabled below this
FW_ALT_MAX	500.0	m	Ceiling — thermal switches to glide above this
FW_ALT_HYST	20.0	m	Hysteresis band below FW_ALT_MAX
FW_POLAR_A	0.003	s/m	Polar: drag coefficient a (sink = $aV^2 + b$)
FW_POLAR_B	0.5	m/s	Polar: minimum sink rate b
FW_GLIDE_I_DECAY	10.0	s	Throttle integrator decay time constant
FW_GLIDE_RAMP_T	2.0	s	Throttle ramp duration on glide entry/exit
FW_THERMAL_BANK	40.0	deg	Default thermal bank angle
